DBMS ASSIGNMENT

ASSIGNMENT 9

Advanced SQL Queries Using Joins and Subqueries

Project: Library Loan Management System

Database: MySQL 8.0

Objective

Use joins, subqueries, derived tables, and correlated subqueries to generate meaningful library management reports.

Note: outputs are expected results based on the included sample data. Run the SQL in your MySQL environment and insert your own screenshots if required by your instructor.

1. Introduction

The reports combine Members, Books, Authors, BookCopies, BookAuthors, and Loans. The overdue report is evaluated as of 22 October 2026.

2. Overdue Report with Member and Book Details

```
SELECT l.LoanID, m.FullName, b.Title,
       DATEDIFF('2026-10-22', l.DueOn) AS DaysOverdue
FROM Loans AS l
JOIN Members AS m ON m.MemberID = l.MemberID
JOIN BookCopies AS c ON c.CopyID = l.CopyID
JOIN Books AS b ON b.BookID = c.BookID
WHERE l.ReturnedOn IS NULL AND l.DueOn < '2026-10-22'
ORDER BY DaysOverdue DESC, l.LoanID;

Expected output:
9002 | Ravi | Database Practice Workbook | 49
9001 | Asha | Pride and Prejudice | 31
9015 | Neha | Python Lab Notes | 3

Explanation: joins convert foreign-key identifiers into a useful overdue follow-up list.
```

3. Borrowing Activity for Every Member

```
SELECT m.MemberID, m.FullName,
       COUNT(l.LoanID) AS TotalLoans,
       SUM(CASE WHEN l.LoanID IS NOT NULL AND l.ReturnedOn IS NULL THEN 1 ELSE 0 END) AS OpenLoans
FROM Members AS m
LEFT JOIN Loans AS l ON l.MemberID = m.MemberID
GROUP BY m.MemberID, m.FullName
ORDER BY m.MemberID;

Expected output:
21 | Asha | 2 | 1
22 | Ravi | 2 | 1
23 | Meera | 0 | 0
24 | Neha | 3 | 1
25 | Arjun | 3 | 1
26 | Priya | 3 | 1
27 | Kabir | 3 | 1
28 | Ishita | 2 | 0
29 | Dev | 2 | 1
30 | Sana | 0 | 0

Explanation: LEFT JOIN retains members with no borrowing history. COUNT(l.LoanID) returns zero for
unmatched rows.
```

4. Available Copies Using NOT EXISTS

```
SELECT c.CopyID, b.Title, c.ShelfLocation
FROM BookCopies AS c
JOIN Books AS b ON b.BookID = c.BookID
WHERE c.ConditionStatus = 'Good'
  AND NOT EXISTS (
      SELECT 1 FROM Loans AS l
      WHERE l.CopyID = c.CopyID AND l.ReturnedOn IS NULL
  )
ORDER BY c.CopyID;

Expected output:
502 | Pride and Prejudice | A2-S3
505 | Database Practice Workbook | B1-S1
507 | Python Lab Notes | B2-S1
511 | The Rainy Platform | A2-S4
513 | Everyday Astronomy | C1-S1
516 | Campus Garden Handbook | C1-S2

Explanation: the correlated subquery excludes copies with an open loan. Six good-condition copies are
available.
```

5. Members Above the Overall Average Loan Count

```
SELECT m.MemberID, m.FullName, COUNT(l.LoanID) AS LoansTaken
FROM Members AS m
LEFT JOIN Loans AS l ON l.MemberID = m.MemberID
GROUP BY m.MemberID, m.FullName
HAVING COUNT(l.LoanID) > (
    SELECT COUNT(*) * 1.0 / NULLIF((SELECT COUNT(*) FROM Members), 0)
    FROM Loans
)
ORDER BY m.MemberID;

Expected output:
24 | Neha | 3
25 | Arjun | 3
26 | Priya | 3
27 | Kabir | 3

Explanation: the average is 20 loans / 10 members = 2.00 loans per member, including non-borrowers.
NULLIF prevents division by zero if the member table is empty.
```

6. Book Popularity and Co-Authors

```
SELECT b.BookID, b.Title, a.AuthorNames,
       COALESCE(x.LoanCount, 0) AS LoanCount
FROM Books AS b
LEFT JOIN (
    SELECT ba.BookID,
           GROUP_CONCAT(a.AuthorName ORDER BY a.AuthorID SEPARATOR ', ') AS AuthorNames
    FROM BookAuthors AS ba
    JOIN Authors AS a ON a.AuthorID = ba.AuthorID
    GROUP BY ba.BookID
) AS a ON a.BookID = b.BookID
LEFT JOIN (
    SELECT c.BookID, COUNT(l.LoanID) AS LoanCount
    FROM BookCopies AS c
    LEFT JOIN Loans AS l ON l.CopyID = c.CopyID
    GROUP BY c.BookID
) AS x ON x.BookID = b.BookID
ORDER BY LoanCount DESC, b.BookID;

Expected output:
103 | Python Lab Notes | Kiran Shah | 4
101 | Pride and Prejudice | Jane Austen | 3
102 | Database Practice Workbook | Arun Rao, Nisha Das | 3
105 | The Rainy Platform | Rohan Sen | 3
106 | Everyday Astronomy | Farah Ali | 3
104 | Networks Made Simple | Kiran Shah, Leela Menon | 2
107 | Statistics Through Examples | Vivek Nair | 2
108 | Campus Garden Handbook | Anita Bose | 0

Explanation: authors and loan counts are aggregated separately before joining so a co-authored book does
not have its loan count doubled.
```

7. Titles Never Borrowed

```
SELECT b.BookID, b.Title
FROM Books AS b
WHERE NOT EXISTS (
    SELECT 1 FROM BookCopies AS c
    JOIN Loans AS l ON l.CopyID = c.CopyID
    WHERE c.BookID = b.BookID
)
ORDER BY b.BookID;

Expected output:
108 | Campus Garden Handbook

Explanation: the query checks loan history across every physical copy of each title. The garden handbook
is the only title never borrowed in the sample.
```

8. Conclusion

The reports demonstrate INNER JOIN, LEFT JOIN, GROUP BY, aggregate functions, NOT EXISTS, scalar subqueries, and derived tables. They support overdue follow-up, circulation monitoring, availability checking, and catalogue popularity analysis.